

Knowledge Gained — Languages, Tools & Tech Stack

🧠 **This is the heart of the report** — a categorised, point-wise inventory of every language, framework, tool and methodology I have learned and shipped with. Each item notes *where* and *how* I applied it, so the depth is verifiable against real work: the **Mark AI** DOOH signage platform I build as sole engineer (**345 commits** · **102 PRs** across 9 repos), **244 pull requests** at VibeMonitor, and **77 personal repositories**.

1 · Programming Languages

Language	Depth & where I applied it
Python	Primary backend language in both roles. At Mark AI — the FastAPI + PostgreSQL signage backend (20+ domain modules, 40 Alembic migrations), a second FastAPI service for the advertiser platform, the AI booking co-pilot, and the Creative Fit image pipeline. At VibeMonitor — FastAPI services, async SQLAlchemy, LangGraph AI agents, AWS / Datadog / New Relic / Azure SDK wrappers, log-ingestion workers, and the Python SDK.
TypeScript	Primary frontend language — the Mark AI operator CMS and the advertiser platform (Next.js 15), the VibeMonitor web app (Next.js + React 19), the JS/TS SDK monorepo, and freelance/personal builds (Sakhi, E-Library).
Kotlin	The Mark AI Android signage player — a production APK that runs fullscreen 24/7 on physical screens: self-enrolment from a baked-in key, scheduling, playback, screenshot capture and proof-of-play, with unit tests over the gating logic.
JavaScript	Node/Express backends (PrepSaarthi), vanilla DOM apps (SipCraft), and Three.js (Cosmic-Trek, the Mark AI 3D city and screen visualisation).
Java	One of the three log-ingestion SDK languages; plus DSA practice & academic coursework.
SQL	PostgreSQL schema design, B-tree indexing, server-side pagination, Alembic migrations (40 at Mark AI); ClickHouse columnar queries for logs.
Go · Dart · C · C++	Exposure — Go (org microservices), Dart (iTax Android APK), C/C++ (academic foundations & DSA).

2 · Backend & API Engineering

Stack: FastAPI · SQLAlchemy 2.0 async · Alembic · Pydantic v2 · asyncpg · Node.js · Express.js · Uvicorn · Poetry · API-key partner APIs

- **FastAPI + layered architecture** — RESTful APIs with a clean *router* → *service* → *schema* → *model* separation: **35+ domain modules** at VibeMonitor and **20+** at Mark AI (screens, bookings, loops, schedule, playback, content, screenshots, api_keys, cms_users, ai, creative_fit).
- **Device-fleet API design (Mark AI)** — a deliberately **stateless, REST-only contract** between the screens and the backend: `register` (idempotent on device UID) · `heartbeat` · `schedule` · `proof`. I *removed* the WebSocket and folded the live channel into the heartbeat (`reload_needed` , `now_playing` , pending screenshot jobs) — no held sockets means the fleet scales horizontally.

- **Scheduling engine (Mark AI)** — four booking modes (continuous, specific-time, times-per-day, custom) merged with per-screen playback loops into one player schedule, with owner-fixed slot/loop durations, IST operating-hour windows validated on both sides, and a schedule that becomes immutable once a campaign is booked.
- **Approval workflows & lifecycle** — a four-part booking-approval flow spanning three services (bookings wait for the screen owner's decision; approval is attributed to the operator who gave it), three booking statuses with a terminal cancel, and automatic completion when a campaign can no longer play.
- **Reporting & exports** — generated campaign **PDF reports**, a full proof-of-play **Excel export**, and signed-URL bundles for milestone evidence frames.
- **Async SQLAlchemy 2.0 + asyncpg** — fully async ORM over PostgreSQL with connection pooling and server-side pagination instead of shipping every row.
- **Alembic migrations** — atomic, reversible migrations (40 on the Mark AI signage schema); added a migration-validation gate to the VibeMonitor CD pipeline so bad migrations never reach production.
- **Auth & sessions** — JWT (PyJWT), HttpOnly refresh-token rotation, "Remember Me", bcrypt/Argon2 hashing, Google & GitHub OAuth 2.0 (with PKCE), Google One-Tap; at Mark AI a unified email-first flow, operator identity by email, and password reset over a single-use emailed link across two front-ends.
- **Multi-tenant authorisation** — role models enforced per resource: superadmin / screenowner with per-screen assignment and full *screenowner parity* across every surface (Mark AI); teams, workspaces & superadmin tooling (VibeMonitor).
- **Resilience & throughput** — slowapi rate limiting, Tenacity retries, Redis pub/sub for real-time streaming, B-tree indexing for ~70% lower query latency (Sakhi), and round-robin load balancing with connection pooling (earlier Mark AI product).
- **Payments & billing** — full Stripe integration (subscriptions, plan up/downgrade, usage limits) and **Razorpay** on the current Mark AI platform, the earlier Mark AI product, and PrepSaarthi.
- **Real-time** — WebSocket & SSE for chat and live test-run streaming; Socket.io (PrepSaarthi). At Mark AI I made the opposite call on purpose — polling-by-heartbeat beat sockets once the device count grew.

3 · Frontend Engineering

Stack: Next.js 15 · React 19 · TypeScript · Tailwind CSS 4 · Radix UI / shadcn · Zustand · TanStack Query · TipTap · Recharts · React Flow (xyflow) · Motion · Three.js / WebGL · Angular · Redux · Vite

- **Next.js 15 (App Router, Turbopack) + React 19** — the VibeMonitor web app, the Mark AI **operator CMS** (fleet monitoring, bookings, loops, content, reports, AI chat) and the Mark AI **advertiser platform** (marketing site, email-first auth, browse screens, booking wizard, dashboard, proof-of-play reports, screenowner console).
- **Domain-specific UI** — a **live slot map** that renders a screen's playback loop as taken / yours / open and previews a slot before booking; four screenshot states so "Capturing..." can actually end; an interactive plays-trend chart; a needs-attention panel that names the screens that should be on but aren't.
- **3D on the web** — **Three.js / WebGL** city and screen-visualisation scenes with Blender-authored assets, plus an AR hologram section, so a client can see a campaign before booking it.

- **Shared logic across the stack** — ported the backend's fuzzy text scorer into the frontend and guarded both copies with a **shared golden table**, so search ranks identically on either side.
- **State management** — led the migration from **Redux** → **Zustand**; server-state with TanStack Query and a lazy-loaded API-factory pattern.
- **Design system** — Tailwind CSS 4 + Radix UI primitives (shadcn-style), dark mode, mobile-first responsive passes (including a full mobile rework of the CMS), framer-motion animations, Error Boundaries.
- **Rich features** — a full **TipTap** blog editor (autosave, tables, code blocks, image upload), **Recharts** dashboards, a **React-Flow** service-dependency graph, and SSE-streamed chat & test replay views.
- **Earlier stacks** — Angular (Legal E-Library), Redux + Vite migration (PrepSaathi, +40% engagement), Material-UI, AOS, EmailJS.

4 · Android & Digital Signage (Mark AI)

Stack: Kotlin · Android SDK · ExoPlayer / Media3 · WorkManager · PixelCopy / SurfaceView · LocationManager / Geocoder · AOSP (no Play Services) · adb / MIUI deployment · JUnit

- **Built a production signage player APK** that plays ads fullscreen 24/7 on physical Android media boxes — self-registers with a permanent device UID, holds the calendar so each ad starts on its own boundary, gates playback by the screen's operating hours, restarts on boot, and **re-registers itself when its token expires instead of going dark**.
- **Zero-touch enrolment & distribution** — a per-screenowner enrollment key, a first-launch gate, **per-owner APK builds with the key already baked in**, a release pipeline, and fleet-wide version reporting so I can see what every screen is running.
- **Proof-of-play** — device-signed playback records tagged with their `booking_id`, posted back as the billing evidence trail.
- **Screenshot evidence** — milestone and on-demand captures fired on the ad's *render callback* (not a timer), read from the `SurfaceView` for video, never on a half-rendered multi-zone layout, never twice for the same milestone, with black-frame detection, a capture guard, orphan sweeping and a WorkManager upload queue that survives connectivity loss. (*A documented limit: these capture our own render buffer, so they prove software-side playback — never panel hardware faults.*)
- **Location on hard devices** — `LocationManager` + built-in `Geocoder` (no API key, no Play Services), a fresh fix taken at capture time and burned into the frame as an overlay, an operator-triggered re-scan delivered over the heartbeat, and a fallback for screens that take their internet from an **Ethernet cable** with no GPS or WiFi to geolocate from.
- **Power-aware behaviour** — outside operating hours the screen goes black and drops to a slow keep-alive heartbeat rather than silent, so a change still reaches a closed screen.
- **Real-device debugging** — deploying to physical boxes over adb, working around vendor installer restrictions, and diagnosing hardware decode limits (4K60 sources that had to be transcoded to 1080p30).

5 · AI / LLM Engineering (Differentiator)

Stack: LangGraph · LangChain · Tool-calling agents · Agent guardrails · LangFuse · LangSmith · OpenAI · Google Gemini · Azure OpenAI · RAG · Playwright · tree-sitter · HolmesGPT · Hunyuan3D · LaMa inpainting

- **Conversational booking co-pilot (Mark AI)** — an agent embedded in the CMS that turns a plain-English request into a real campaign booking: tool-calling over the live domain (screens, campaigns, availability, proof-of-play), a draft → confirm loop, **two layers of guardrails**, and a **deterministic slot-capture backstop** so it never re-asks for a detail the operator already gave.
- **Grounding an agent in reality** — a current-datetime tool plus per-turn IST context (bookings were landing in the wrong year), per-screenowner scoping on both read tools and booking executors, and operating-hours-aware validation before a booking is accepted.
- **Retrieval & ranking I wrote myself (Mark AI)** — a **typo-tolerant text scorer** matching on word coverage rather than a weighted average, **haversine distance** ranking with a padded fleet bounding box and forward geocoding, explicit distance bands taught to the agent, locality-misspelling correction against the real fleet, capped result counts and an "ask when unsure" path instead of guessing.
- **Domain analysis tools** — campaign lookup by name, date-range listing, and strict proof-of-play gap & health analysis so the agent can answer "did this ad actually play?" without inventing an answer.
- **Agent evaluation without unit tests** — verified the co-pilot with an **actor/verifier agent harness** driving real chat sessions against a seeded database, written up as a test report.
- **Generative creative (Mark AI "Creative Fit")** — rebuilds a finished advertisement for any screen resolution: a Gemini-generated layout blueprint, **LaMa** inpainting to erase original elements, and a Pillow relay layout pass — output kept native/4K because these play on physical premium screens.
- **Generative 3D (Mark AI)** — image/text → 3D asset generation with a self-hosted **Hunyuan3D** on a Cloud Run **L4 GPU** service, feeding a web AR preview.
- **Agentic RCA engine (LangGraph, VibeMonitor)** — an automated Root-Cause-Analysis agent that traces incidents through commit history and code changes; wraps CloudWatch, New Relic, Datadog & Azure tools as agent tools.
- **BYOLLM** — "Bring Your Own LLM": pluggable provider layer over OpenAI, Azure OpenAI and Gemini with credential verification.
- **LLM observability** — LangFuse / LangSmith trace reading, prompt engineering, LLM config & model-version management.
- **RAG** — a chatbot grounded strictly in a source document (Cosmic-Trek's Orbital-Mechanics chatbot).
- **Code intelligence** — tree-sitter parsing to let the agent reason over source code during RCA.
- **AI test generation (Beryl)** — natural-language → Playwright script conversion, self-healing selectors, auto-retry with NL-refactor regeneration.
- **Evaluation** — implemented the **HolmesGPT** benchmark dataset and reached ~90% accuracy on the first 10–20 RCA test cases.

6 · Databases & Storage

Stack: PostgreSQL · Cloud SQL · ClickHouse · MongoDB · Redis · Firebase / Firestore · Prisma · Mongoose · AWS S3 · GCS + signed URLs

- **PostgreSQL** — primary store in both roles. At **Mark AI** it is the fleet's single source of truth (screens, bookings, loops, proof-of-play, screenshots) across 40 Alembic migrations on **Cloud SQL**, with server-side pagination and a capped instance count so the connection pool can't be exhausted. Earlier: a schema redesign that cut query times ~60% (iTax).
- **ClickHouse** — columnar store for the high-throughput log-ingestion platform; tuned batching, compression & bounded time-range queries.

- **Redis** — pub/sub for real-time job/result streaming in the Beryl pipeline.
- **Object storage** — S3 for blog images; Google Cloud Storage with **signed URLs** for Mark AI media, screenshots and generated 3D assets (no public buckets, per org policy), with creative uploads going **direct to GCS** rather than proxied through the API.
- **MongoDB + Mongoose** — PrepSaarthi, Sakhi, and the earlier Mark AI product; indexing for 1.5–2k concurrent users. (*The current Mark AI platform is PostgreSQL-based.*)
- **Firebase / Firestore** — real-time data & auth (UpInTheSky).
- **ORMs** — Prisma (Next.js projects), SQLAlchemy (Python), Mongoose (Node).

7 · Observability & Monitoring (Core Domain)

Stack: Prometheus · Grafana · Datadog · New Relic APM · AWS CloudWatch · OpenTelemetry · Loki · Tempo · Mimir · SigNoz · Sentry · Azure Monitor · GCP Monitoring · Device-fleet health

- **Unified observability platform** — correlated logs, metrics & traces across CloudWatch, New Relic, Datadog, Grafana, Azure Monitor & GCP for 15+ microservices, with RBAC and custom pipelines.
- **Instrumentation** — custom Prometheus metrics & alert rules; OpenTelemetry (OTLP) exporters & auto-instrumentation; Sentry error tracking.
- **Applied it to hardware (Mark AI)** — a screen-health view built on the same thinking: per-device capability tiers, heartbeat-carried device stats, fleet version reporting, a needs-attention panel that names screens that should be on but aren't, and an *off-hours* state kept distinct from *offline* so a screen closed for the night never pages anyone.
- **Methodologies** — applied **RED** (Rate, Errors, Duration) and **USE** (Utilization, Saturation, Errors) to build service dashboards; wrote an internal blog on both.
- **Alerting gaps** — built a feature that discovers missing RDS/PostgreSQL CloudWatch alert rules and recommends them.

8 · Cloud, DevOps & Infrastructure

Stack: AWS (EKS · ECS · RDS · IAM · CloudWatch · S3) · GCP (Cloud Run · Cloud SQL · GCS) · Cloud Run GPU (L4) · Workload Identity Federation · GitOps config apply · Azure · Docker · Kubernetes · GitHub Actions CI/CD · Alembic CD gate · hPanel

- **AWS** — container orchestration on EKS, production services on ECS (task definitions), RDS (automated snapshot management & migrations), IAM roles/trust policies, CloudWatch, S3.
- **GCP (Mark AI)** — the whole platform runs on **Cloud Run** in front of **Cloud SQL**, with Cloud Build image pipelines, custom domains, capped instance counts to protect the database, and a separate **GPU (L4) Cloud Run service** for 3D generation. Worked inside real org-policy constraints: **Workload Identity Federation** instead of service-account keys, signed URLs instead of public buckets, and an explicit unauthenticated-invoker exception for public services.
- **Config as code** — a dedicated secrets/infra repository whose CI **applies service configuration to Cloud Run on push to `main`**, with a read-only diff step that reports drift without failing the apply.
- **Release engineering** — an Android **APK release pipeline** producing per-owner signed builds with an enrolment key embedded, distributed from the CMS, with the fleet reporting back which version each screen runs.

- **Containers & orchestration** — Docker / docker-compose, Kubernetes (EKS) service scaling & multi-environment deploys.
- **CI/CD** — multi-stage GitHub Actions with build caching & artifact reuse ($\approx 20\%$ faster builds across 10+ pipelines), deployment notifications, change-detection rebuilds, buildx cache tuning, and scheduled cron jobs.
- **Multi-environment** — dev / preview / production deploys, plus a full local stack (Docker Postgres + Poetry + Android emulator) for the Mark AI system.
- **Earlier hosting** — full server/deployment ownership on hPanel with load balancing (iTax, PrepSaarthi).

9 · Security & Compliance

Stack: SOC 2 work · Dependabot · pip-audit · Bandit · Gitleaks · Presidio PII redaction · SSRF guard · Cloudflare Turnstile · Credential encryption · Signed device payloads

- **Vulnerability management** — recurring Dependabot remediation, pip-audit & Bandit scanning, secret scanning with Gitleaks — driven by **SOC 2** compliance.
- **Data protection** — PII detection & redaction in logs with Presidio + spaCy; encryption of stored integration credentials.
- **App security** — server-side **SSRF guard** for crawl URLs, Cloudflare **Turnstile** captcha, HttpOnly cookie sessions, API-key rate limiting, single-use password-reset links, and "keys never visible in the frontend" hardening (no secrets in `NEXT_PUBLIC_*`).
- **Untrusted-device security (Mark AI)** — screens live in third-party venues, so the player holds only a scoped enrolment credential, signs its proof-of-play payloads, re-registers rather than retaining a stale token, and every partner call goes through a revocable API key with per-key attribution.
- **Engineering priority learned:** customer-data security comes first, then speed, then future-proofing.

10 · Integrations Built (third-party)

Stack: GitHub App / OAuth / Webhooks · Slack · Grafana · Datadog · New Relic · AWS CloudWatch · Azure Monitor · GCP · Kubernetes · Stripe · Razorpay · Mailgun → Postmark · Gmail API · Twilio WhatsApp · PostHog · Gemini API · MindAR · Xibo (legacy) · Sandbox GST/ITR API

I became the team's go-to engineer for integrations — owning the full wrapper pattern (connect → verify → store-encrypted → health-check → disconnect) so new providers could be added quickly and documented.

11 · Developer Tools, Practices & Methodologies

Stack: Git / GitHub Apps · Linear · ClickUp · Notion · Confluence / Jira · incident.io · Postman / Swagger · Prisma Studio · pgAdmin · Android Studio · pre-commit · Ruff / Black / isort / mypy · AI-assisted dev (CLAUDE.md)

- **Quality tooling** — pre-commit hooks, Ruff/Black/isort linting & formatting, mypy typing, pytest + pytest-asyncio + coverage; Vitest on the Mark AI CMS, JUnit on the Android player, and golden-table tests shared across two languages.
- **Working solo at pace** — at Mark AI I'm the only engineer, so I still work in small reviewable PRs (102 of them) with conventional commit messages that describe the *behaviour* change, one branch per feature, and a written design doc before any non-trivial build.

- **Workflow** — Linear/ClickUp for tickets, PR-review culture at VibeMonitor (peer reviews to cut PR latency), Postman/Swagger for API contracts, docs-first planning before coding.
- **AI-augmented engineering** — using CLAUDE.md to encode project conventions across a multi-repo system, and ChatGPT/Perplexity for research — while owning the result and never over-building.

12 · Engineering Maturity & Soft Skills

- **Bias for action** — act decisively under uncertainty instead of over-analysing.
- **Edge-case / QA muscle** — test in every environment after deploying; hunt bugs proactively and file them.
- **Ownership mindset** — own outcomes end-to-end; "you own the result." At Mark AI that means an entire product surface — backend, device, two front-ends and the infrastructure underneath — with no one else to hand a layer to.
- **Knowing what to delete** — the WebSocket, the proxied upload endpoint and the Delete button on bookings were all removed on purpose, and the system got simpler and more correct each time.
- **Consistency & intensity** — repeatedly highlighted as a core strength.
- **Communication & docs** — writing structured documents to bring clarity; an active growth area I'm investing in.